M3. Student Discount & Fine Ledger

Category B Practice - Problem 3 (Intermediate)

Approach

LibraryMember records each applied fine in a private array. StudentMember overrides chargeFine with @Override, halves the amount, and delegates to super. Defensive copies stop callers from changing stored history.

Java Solution

```
import java.util.Arrays;
class LibraryMember {
    protected final String memberId; protected final int borrowLimit;
    private final int[] fineHistory=new int[10]; private int fineCount;
    public LibraryMember(String id,int limit) {
        if(id==null||id.trim().length()<4) throw new IllegalArgumentException("Invalid ID");
        if(limit<=0) throw new IllegalArgumentException("Limit must be positive");
        memberId=id; borrowLimit=limit;
    }
    protected void chargeFine(int amount) {
        if(amount<=0) throw new IllegalArgumentException("Fine must be positive");
        if(fineCount==fineHistory.length) throw new IllegalStateException("Fine history full");
        fineHistory[fineCount++]=amount;
    }
    public int[] getFineHistory() { return Arrays.copyOf(fineHistory,fineCount); }
    public int getTotalFine() { int total=0; for(int i=0;i<fineCount;i++) total+=fineHistory[i]; return total; }
}
class StudentMember extends LibraryMember {
    private final String course;
    public StudentMember(String id,int limit,String course) { super(id,limit); this.course=course; }
    @Override protected void chargeFine(int amount) { super.chargeFine(amount/2); }
}
class FineDemo {
    public static void main(String[] args) {
        StudentMember s=new StudentMember("STU5",3,"CSE"); s.chargeFine(100);
        int[] history=s.getFineHistory(); history[0]=999;
        System.out.println(s.getTotalFine());
        System.out.println(Arrays.toString(s.getFineHistory()));
    }
}
```

Sample Output

```
50
[50]
```